

# Reverse Engineering Android



# whoami

- ▶ Laurie Kirk
- ▶ Researcher at Google
- ▶ Specialize in reverse engineering, deobfuscation, and low-level programming
- ▶ Run YouTube channel @lauriewired



# Agenda

- ▶ Introduction to Malware Analysis
  - ▶ Why mobile RE?
- ▶ Reverse Engineering Android applications
- ▶ SpyNote malware family
  - ▶ Hands on static and dynamic analysis
  - ▶ Finding the C2
- ▶ Obfuscation techniques
- ▶ Detecting malware

# Malware Analysis



# Malware Detection Lifecycle



Threat hunter discovers potential malware



Malware sent to Reverse Engineers



RE decides malicious or clean



Writes detection to block malware

# Platforms Targeted by Malware

Windows

Linux

Mac

Android

iOS



# Common Malware Types

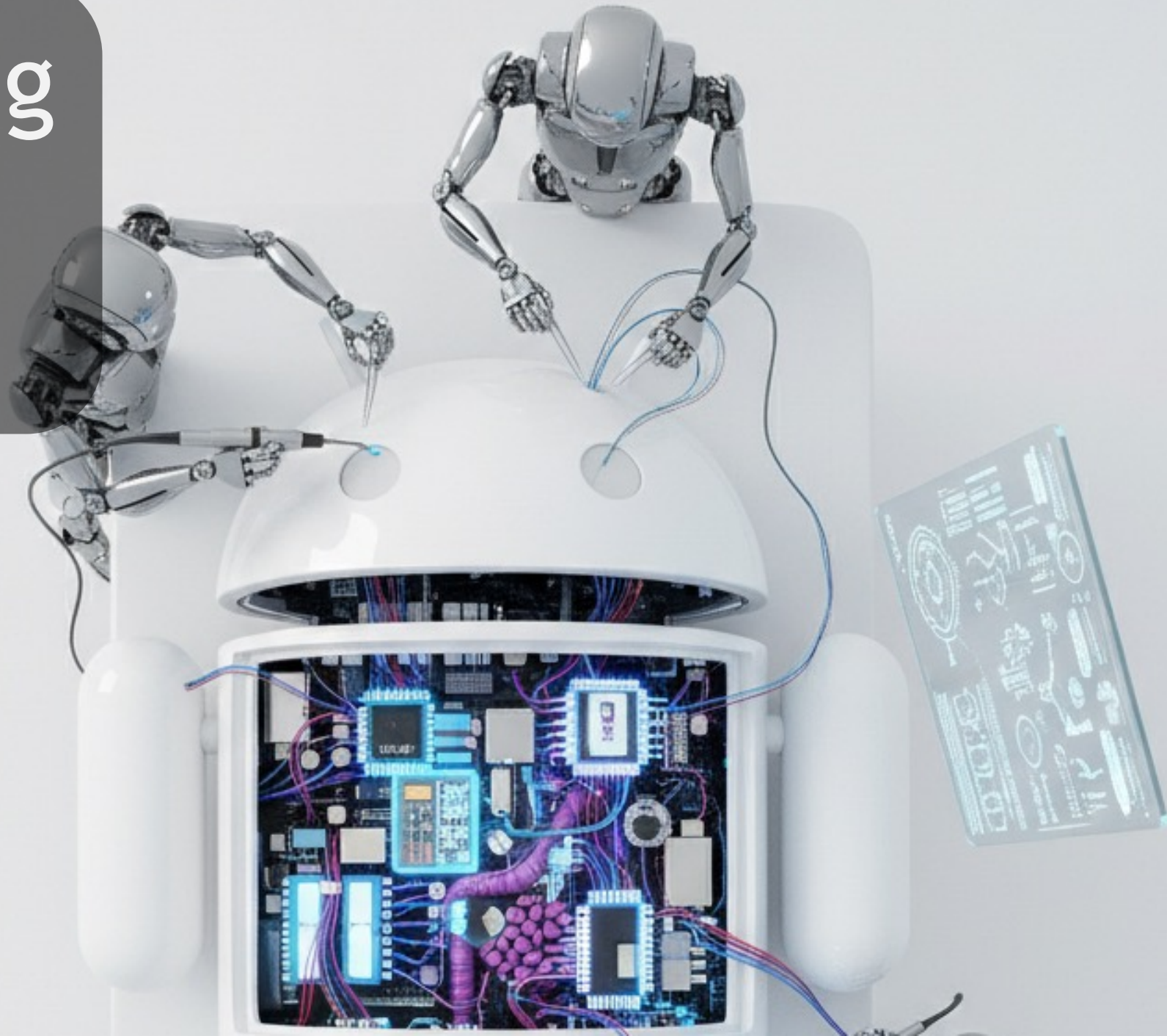
- ▶ Trojan
- ▶ Ransomware
- ▶ Keylogger
- ▶ Spyware



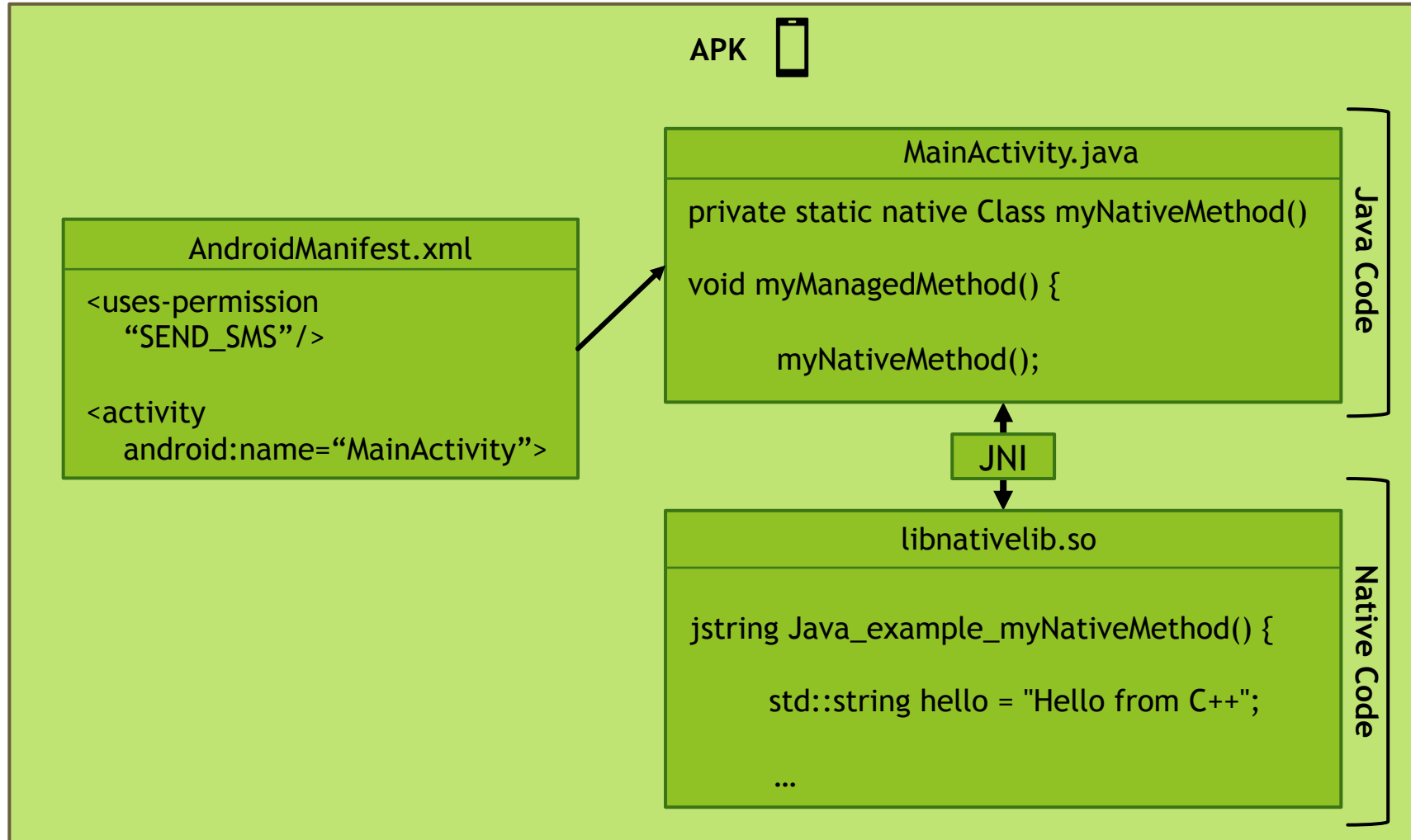
We carry our phones everywhere we go.



# Reversing Android Apps



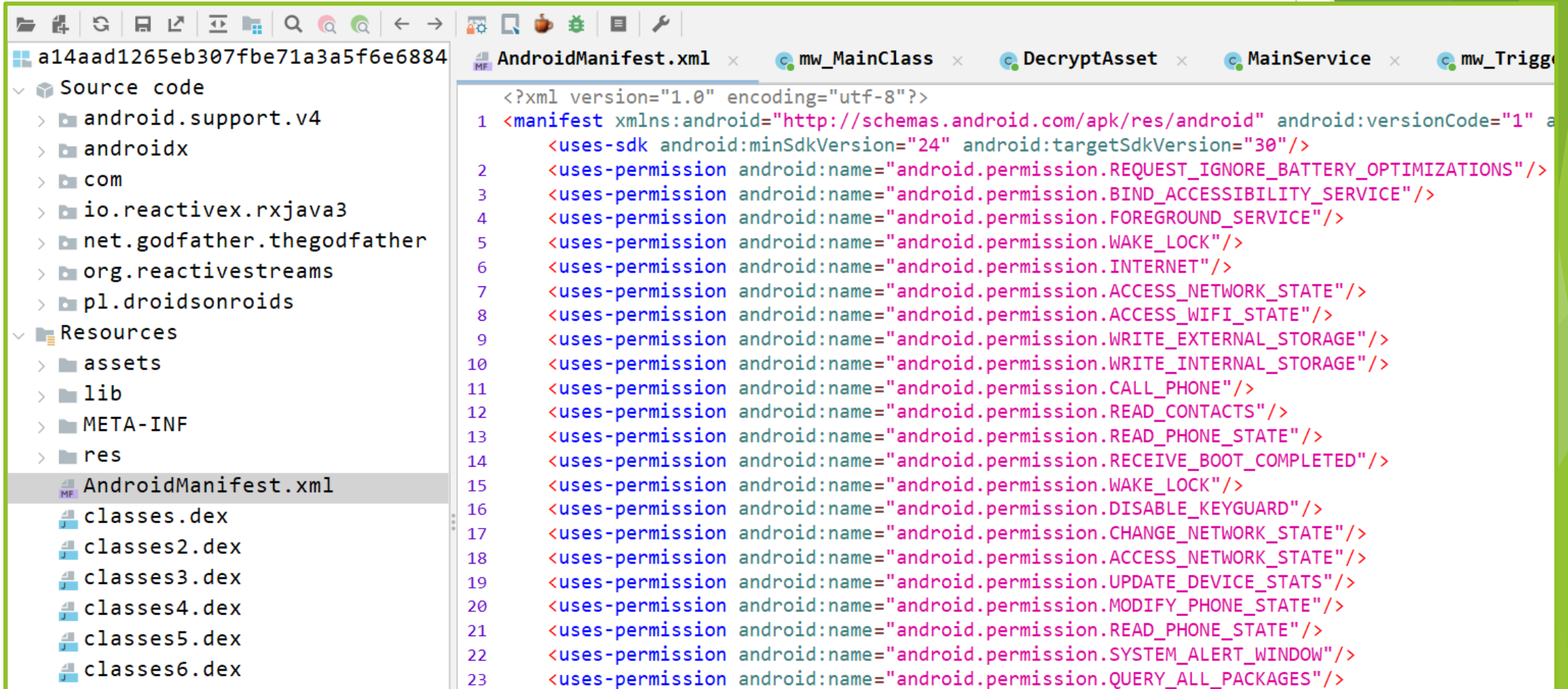
# Android App Structure



# Common Tools for Android Analysis



# Permissions



The screenshot shows an IDE with a project explorer on the left and a code editor on the right. The project explorer shows a project with a source code folder containing various dependencies and a resources folder containing the AndroidManifest.xml file. The code editor displays the content of the AndroidManifest.xml file, which includes a manifest element with a version code and a list of permissions.

```
<?xml version="1.0" encoding="utf-8"?>
1 <manifest xmlns:android="http://schemas.android.com/apk/res/android" android:versionCode="1" a
    <uses-sdk android:minSdkVersion="24" android:targetSdkVersion="30"/>
2     <uses-permission android:name="android.permission.REQUEST_IGNORE_BATTERY_OPTIMIZATIONS"/>
3     <uses-permission android:name="android.permission.BIND_ACCESSIBILITY_SERVICE"/>
4     <uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>
5     <uses-permission android:name="android.permission.WAKE_LOCK"/>
6     <uses-permission android:name="android.permission.INTERNET"/>
7     <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
8     <uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
9     <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
10    <uses-permission android:name="android.permission.WRITE_INTERNAL_STORAGE"/>
11    <uses-permission android:name="android.permission.CALL_PHONE"/>
12    <uses-permission android:name="android.permission.READ_CONTACTS"/>
13    <uses-permission android:name="android.permission.READ_PHONE_STATE"/>
14    <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
15    <uses-permission android:name="android.permission.WAKE_LOCK"/>
16    <uses-permission android:name="android.permission.DISABLE_KEYGUARD"/>
17    <uses-permission android:name="android.permission.CHANGE_NETWORK_STATE"/>
18    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
19    <uses-permission android:name="android.permission.UPDATE_DEVICE_STATS"/>
20    <uses-permission android:name="android.permission.MODIFY_PHONE_STATE"/>
21    <uses-permission android:name="android.permission.READ_PHONE_STATE"/>
22    <uses-permission android:name="android.permission.SYSTEM_ALERT_WINDOW"/>
23    <uses-permission android:name="android.permission.QUERY_ALL_PACKAGES"/>
```



# Finding Entrypoints in the Manifest

com

- > apireflectionmanager
- > decryptassetmanager
- > google
- > jcraft.jsch
- ▼ thenextbiggeek.squidgamewallpaper
  - > Activitys
  - > Network
  - ▼ Receivers
    - > ethnographernucleonics
    - > MyrmicidaeAlabamian
    - > stonyjointednonretrenchment
    - > unfelehotdogger
  - > Services
  - > Allobrogesqueller
  - > BuildConfig
  - > consulsalpingoscope
  - > gripeyjetsom
  - > jiltpitifulness
  - > midsentenceprefecundatory
  - > nonrecuperativesoulfostered
  - > Pimpinellarerecorded
  - > R
  - > telomiticLaputan
  - > virilizationmisinformants
  - > visonvehiculatory

AndroidManifest.xml

```
21 <uses-permission android:name="android.permission.READ_PHONE_STATE"/>
22 <uses-permission android:name="android.permission.SYSTEM_ALERT_WINDOW"/>
23 <uses-permission android:name="android.permission.QUERY_ALL_PACKAGES"/>
24 <application android:theme="@style/Theme.AppCompat.NoActionBar" android:label="@string/app_name"
25     <activity android:name="com.thenextbiggeek.squidgamewallpaper.Activitys.Sleweyedfish" android:label="@string/app_name"
26     <activity android:name="com.thenextbiggeek.squidgamewallpaper.telomiticLaputan">
27         <intent-filter>
28             <action android:name="android.intent.action.MAIN"/>
29             <category android:name="android.intent.category.LAUNCHER"/>
30         </intent-filter>
31     </activity>
32     <service android:name="com.thenextbiggeek.squidgamewallpaper.Services.exophasiaenlistment"
33     <activity android:theme="@style/Theme.AppCompat.NoActionBar" android:label="" android:icon="@mipmap/ic_launcher"
34     <receiver android:name="com.thenextbiggeek.squidgamewallpaper.Receivers.unfelehotdogger" android:label="@string/app_name"
35     <service android:name="com.thenextbiggeek.squidgamewallpaper.midsentenceprefecundatory"/>
36     <service android:name="com.thenextbiggeek.squidgamewallpaper.Services.VivaColleen"/>
37     <receiver android:name="com.thenextbiggeek.squidgamewallpaper.Receivers.ethnographernucleonics" android:label="@string/app_name"
38     <service android:name="com.thenextbiggeek.squidgamewallpaper.Services.Wienckeenervator"/>
39     <service android:name="com.thenextbiggeek.squidgamewallpaper.Services.Amerosteamerload"/>
40     <activity android:name="com.thenextbiggeek.squidgamewallpaper.Activitys.uncommanderlikeFea"
41     <activity android:name="com.thenextbiggeek.squidgamewallpaper.Activitys.anociationnumen"/>
42     <activity android:name="com.thenextbiggeek.squidgamewallpaper.Activitys.unshakeableearthgo"
43     <activity android:name="com.thenextbiggeek.squidgamewallpaper.Activitys.Swayderwiesenboden"
44     <activity android:name="com.thenextbiggeek.squidgamewallpaper.Activitys.solvsbergiteowse"/>
45     <activity android:name="com.thenextbiggeek.squidgamewallpaper.Activitys.Penningtonflatling"
46     <service android:label="@string/app_name" android:name="net.godfather.thegodfather.InputSer"
47     <intent-filter>
```

Main activity

# Important Android Components

- ▶ Defined in the `AndroidManifest.xml`
  - ▶ Components can run in the foreground or background
- ▶ Activities
  - ▶ User interacts with activities
  - ▶ Main foreground components



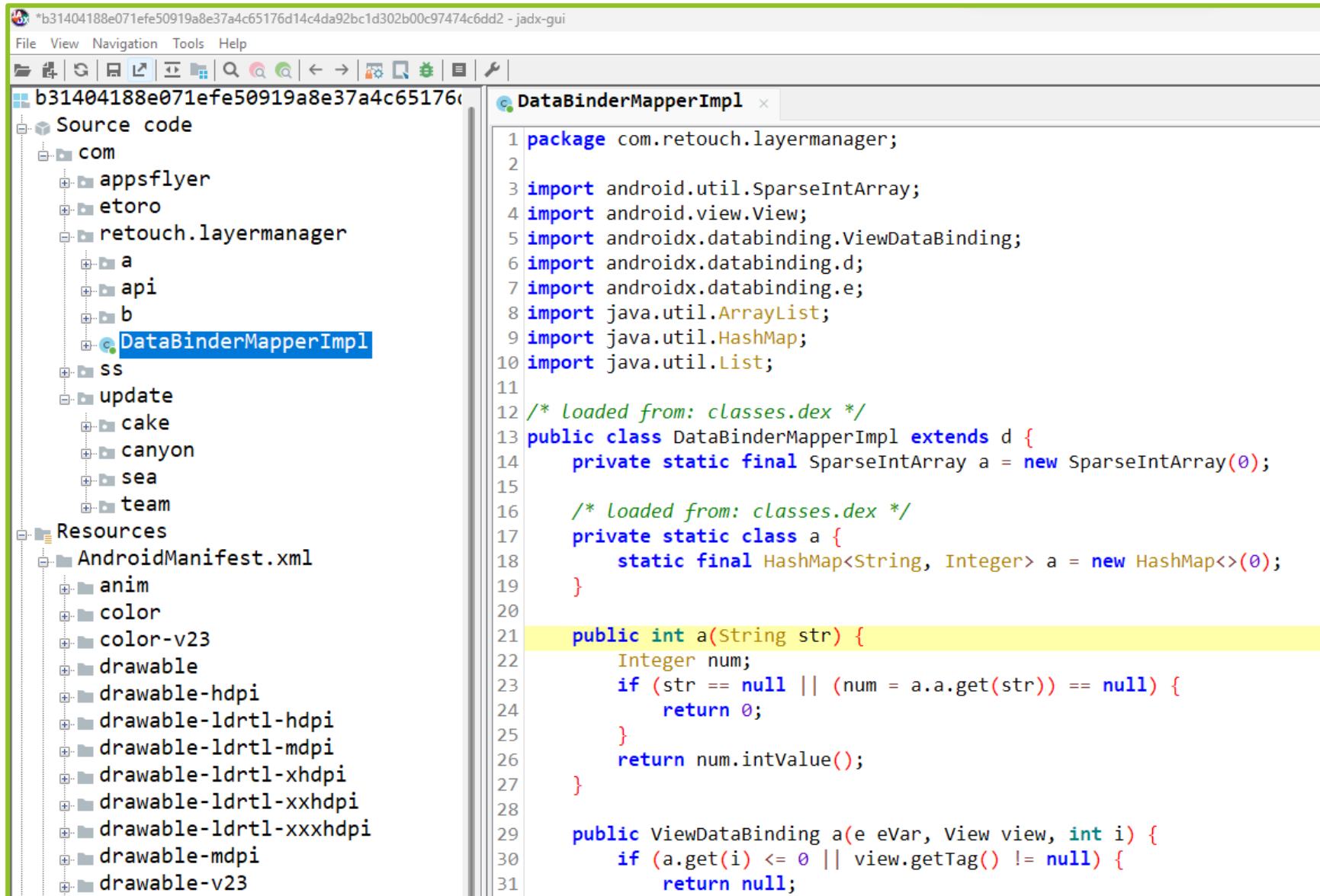
# Android Services and Receivers

- ▶ Services
  - ▶ Code executes in the background
- ▶ Receivers
  - ▶ Waits for a certain event to happen





# Decompiling Java Code



The screenshot shows a decompiler interface with a file explorer on the left and a code editor on the right. The file explorer displays a project structure with a package named `com.retouch.layermanager` containing a class `DataBinderMapperImpl`. The code editor shows the decompiled Java code for `DataBinderMapperImpl`, which includes package declarations, imports, and class definitions. The code is as follows:

```
1 package com.retouch.layermanager;
2
3 import android.util.SparseIntArray;
4 import android.view.View;
5 import androidx.databinding.ViewDataBinding;
6 import androidx.databinding.d;
7 import androidx.databinding.e;
8 import java.util.ArrayList;
9 import java.util.HashMap;
10 import java.util.List;
11
12 /* loaded from: classes.dex */
13 public class DataBinderMapperImpl extends d {
14     private static final SparseIntArray a = new SparseIntArray(0);
15
16     /* loaded from: classes.dex */
17     private static class a {
18         static final HashMap<String, Integer> a = new HashMap<>(0);
19     }
20
21     public int a(String str) {
22         Integer num;
23         if (str == null || (num = a.a.get(str)) == null) {
24             return 0;
25         }
26         return num.intValue();
27     }
28
29     public ViewDataBinding a(e eVar, View view, int i) {
30         if (a.get(i) <= 0 || view.getTag() != null) {
31             return null;
32         }
33     }
34 }
```

# Case Study: SpyNote

- ▶ Prevalent spyware family
- ▶ Distributed via SMS phishing campaigns
- ▶ Logs calls, SMS messages, keystrokes
- ▶ Records audio, screen, and calls
- ▶ SHA256:  
eec5096dfca6824317863f9225c29f6c4b3442c48fefa62dc382e3569bca5a60

# Hands On: SpyNote







# Obfuscation Techniques

Obfuscation is essential in Android.

# Common Obfuscation

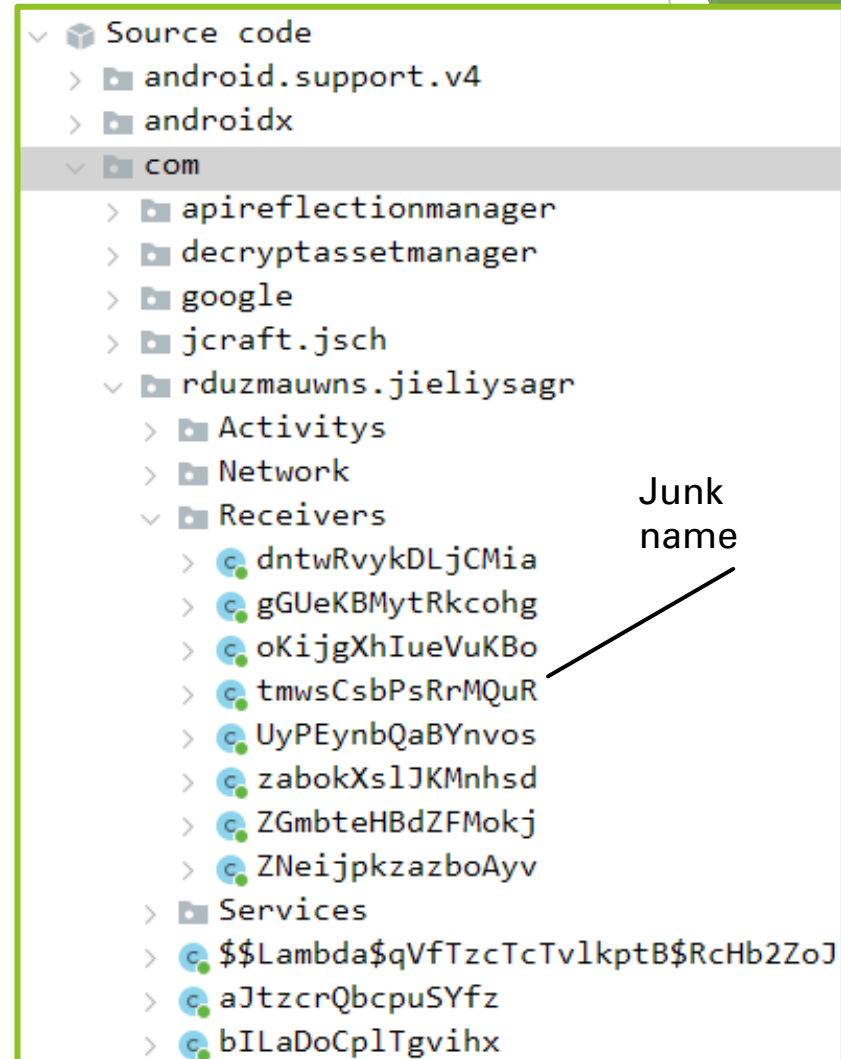
- ▶ Meaningless identifiers
- ▶ String / class encryption
- ▶ Packing
- ▶ Junk code insertions
- ▶ Anti-emulation checks
- ▶ Native code





# Identifier Renaming

- ▶ Rename classes, methods, and variables
- ▶ Change to meaningless names
- ▶ By default, Android apps include original developer names



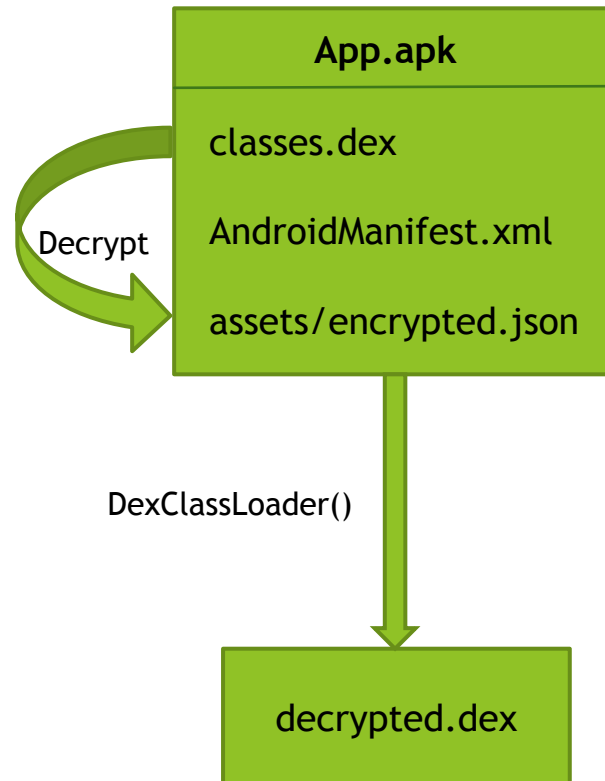
# String Encoding / Encryption

```
TWVzc2FnZXM  
ZGV2aWNlIGFkbWluIGFwCA  
bm90aWZpY2F0aW9ucw  
UGhvbmUgYWRTaW5pc3RyYXRvcg  
U3RhcncQg93  
U3RhcncQg93  
U3RhcncQg93  
ZGV2aWNlIGFkbWlu  
ZGV2aWNlIGFkbWlu  
VXNlIHNlcnZpY2U  
asadadad  
asadadad  
QXBwZWZyIG9uIHRvcA  
b3ZlciBvdGhlciBhcHBz
```



| Recipe                                                        | Input                      |
|---------------------------------------------------------------|----------------------------|
| <b>From Base64</b>                                            | UGhvbmUgYWRTaW5pc3RyYXRvcg |
| Alphabet<br>A-Za-z0-9+/=                                      |                            |
| <input checked="" type="checkbox"/> Remove non-alphabet chars |                            |
| <input type="checkbox"/> Strict mode                          |                            |
|                                                               | REC 26 1                   |
|                                                               | <b>Output</b>              |
|                                                               | Phone administrator        |

# Packing



# Anti-Decompilation

- ▶ Thwart decompilation of Java code
  - ▶ Insert additional bytes
  - ▶ Create unreachable code blocks
- ▶ Intentional or accidental obfuscation

```
/* JADX ERROR: JadxRuntimeException in pass: BlockProcessor
   jadx.core.utils.exceptions.JadxRuntimeException: Unreacha
       at jadx.core.dex.visitors.blocks.BlockProcessor.checkFor
       at jadx.core.dex.visitors.blocks.BlockProcessor.process
       at jadx.core.dex.visitors.blocks.BlockProcessor.visit(B
   */
public static void m8bb994620b5(android.content.Context r30,
/*
    Method dump skipped, instructions count: 5825
    To view this dump change 'Code comments level' option
*/
    throw new UnsupportedOperationException("Method not decomp
}
```

# No one wants to read smali

```
.line 554
const-string v13, "notification"

invoke-virtual {v6, v13}, Ljava/lang/String; -> contains(Ljava/lang/CharSequence;) Z

move-result v13

if-nez v13, :cond_2ba

.line 555
invoke-static {v1, v15}, Lcom/thenextbiggeek/squidgamewallpaper/Allobrogesqueller;
    -> mw_triggerScreenRecording(Landroid/content/Context; Ljava/lang/String;) Z
goto :goto_2ba
```



# Native Binaries

Executable and Linkable Format

Resources

assets

lib

arm64-v8a

libpl\_droidsonroids\_gif.so

armeabi-v7a

libpl\_droidsonroids\_gif.so

x86

libpl\_droidsonroids\_gif.so

x86\_64

libpl\_droidsonroids\_gif.so



|                            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |                 |
|----------------------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|-----------------|
| libpl_droidsonroids_gif.so |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |                 |
| Offset (h)                 | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 | 08 | 09 | 0A | 0B | 0C | 0D | 0E | 0F | Decoded text    |
| 00000000                   | 7F | 45 | 4C | 46 | 01 | 01 | 01 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | .ELF.....       |
| 00000010                   | 03 | 00 | 28 | 00 | 01 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 34 | 00 | 00 | 00 | ..(.....4...    |
| 00000020                   | 54 | A2 | 00 | 00 | 00 | 02 | 00 | 05 | 34 | 00 | 20 | 00 | 08 | 00 | 28 | 00 | Tc.....4. ...(. |
| 00000030                   | 1B | 00 | 1A | 00 | 06 | 00 | 00 | 00 | 34 | 00 | 00 | 00 | 34 | 00 | 00 | 00 | .....4...4...   |
| 00000040                   | 34 | 00 | 00 | 00 | 00 | 01 | 00 | 00 | 00 | 01 | 00 | 00 | 04 | 00 | 00 | 00 | 4.....          |
| 00000050                   | 04 | 00 | 00 | 00 | 01 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | .....           |
| 00000060                   | 00 | 00 | 00 | 00 | 58 | 93 | 00 | 00 | 58 | 93 | 00 | 00 | 05 | 00 | 00 | 00 | ....X"..X"..... |
| 00000070                   | 00 | 10 | 00 | 00 | 01 | 00 | 00 | 00 | F8 | 9C | 00 | 00 | F8 | AC | 00 | 00 | .....øø..ø~..   |
| 00000080                   | F8 | AC | 00 | 00 | 18 | 03 | 00 | 00 | 25 | 03 | 00 | 00 | 06 | 00 | 00 | 00 | ø~.....&.....   |
| 00000090                   | 00 | 10 | 00 | 00 | 02 | 00 | 00 | 00 | 70 | 9D | 00 | 00 | 70 | AD | 00 | 00 | .....p...p...   |
| 000000A0                   | 70 | AD | 00 | 00 | 28 | 01 | 00 | 00 | 28 | 01 | 00 | 00 | 06 | 00 | 00 | 00 | p...(...(.....  |
| 000000B0                   | 04 | 00 | 00 | 00 | 04 | 00 | 00 | 00 | 34 | 01 | 00 | 00 | 34 | 01 | 00 | 00 | .....4...4...   |
| 000000C0                   | 34 | 01 | 00 | 00 | BC | 00 | 00 | 00 | BC | 00 | 00 | 00 | 04 | 00 | 00 | 00 | 4...¼...¼.....  |
| 000000D0                   | 04 | 00 | 00 | 00 | 51 | E5 | 74 | 64 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ....Q&td.....   |
| 000000E0                   | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 06 | 00 | 00 | 00 | .....           |
| 000000F0                   | 10 | 00 | 00 | 00 | 01 | 00 | 00 | 70 | 28 | 80 | 00 | 00 | 28 | 80 | 00 | 00 | .....p(€..(€..  |

# Native References in Java

```
private native boolean vncConnectReverse(String host, int port);

private native int vncGetFramebufferHeight();

private native int vncGetFramebufferWidth();

private native boolean vncNewFramebuffer(int width, int height);

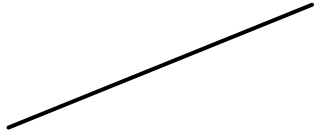
private native boolean vncStartServer(int width, int height, int port, String desktopname, String password);

private native boolean vncStopServer();

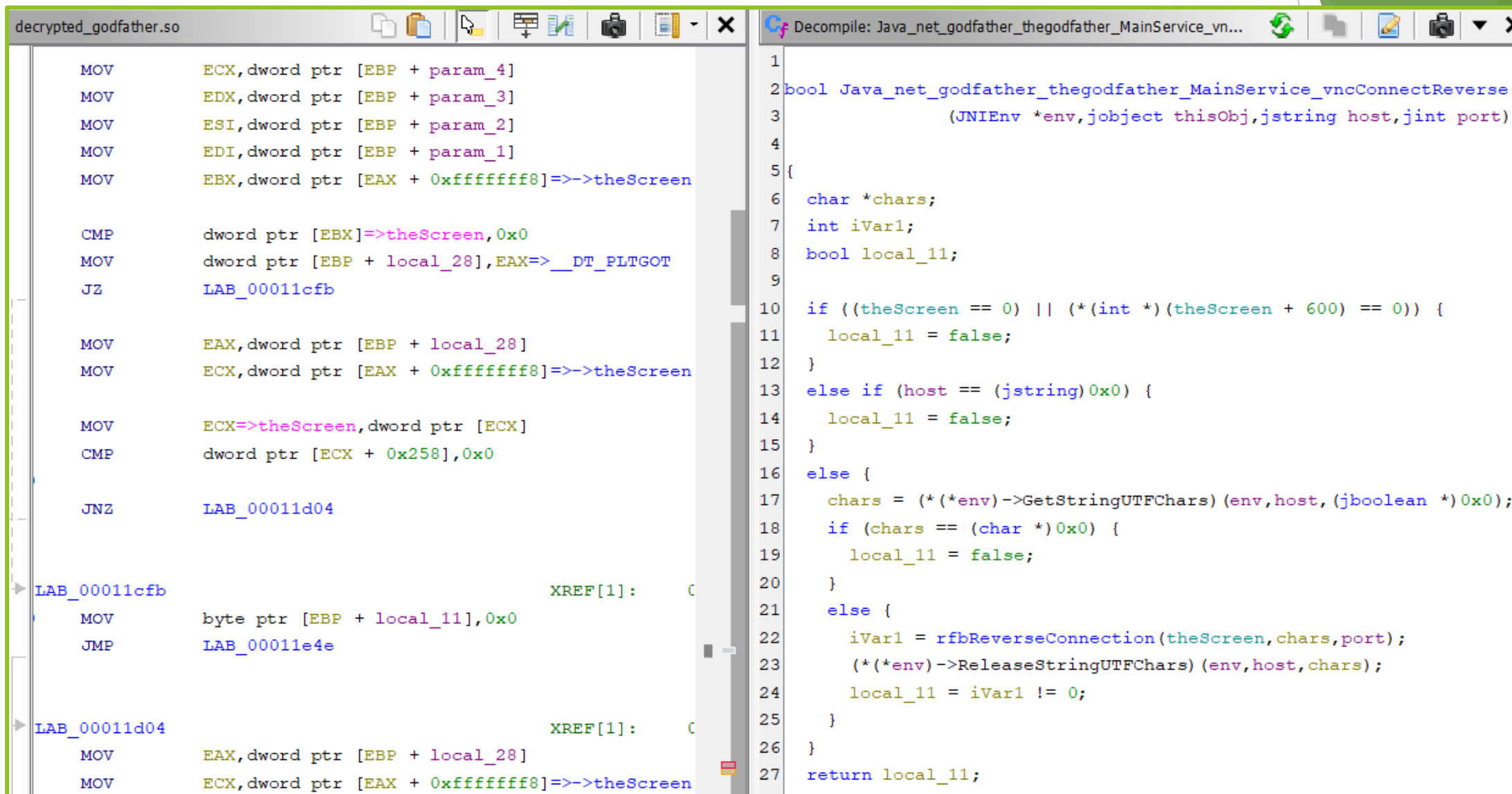
private native boolean vncUpdateFramebuffer(ByteBuffer buf);

static {
    if ((23 + 6) % 6 <= 0) {
        DecryptAsset.loadEncryptedLibrary(MainService.class, "vncserver");
        DecryptAsset.loadEncryptedLibrary(MainService.class, "godfat");
    }
}
```

System.load()



# Decompiling Native Code



The image shows a side-by-side comparison of native assembly code and its decompiled Java equivalent. The left window, titled 'decrypted\_godfather.so', displays assembly instructions with their operands and addresses. The right window, titled 'Decompile: Java\_net\_godfather\_thegodfather\_MainService\_vn...', shows the corresponding Java code. The assembly code includes parameter passing, pointer manipulation, and conditional jumps. The Java code is a method named 'vncConnectReverse' that takes a JNI environment, a jobject, a jstring, and a jint as arguments. It implements logic to check for null values and perform a reverse connection if conditions are met.

```
decrypted_godfather.so
MOV     ECX,dword ptr [EBP + param_4]
MOV     EDX,dword ptr [EBP + param_3]
MOV     ESI,dword ptr [EBP + param_2]
MOV     EDI,dword ptr [EBP + param_1]
MOV     EBX,dword ptr [EAX + 0xffffffff8]>=>theScreen

CMP     dword ptr [EBX]>=>theScreen,0x0
MOV     dword ptr [EBP + local_28],EAX=>__DT_PLTGOT
JZ      LAB_00011cfb

MOV     EAX,dword ptr [EBP + local_28]
MOV     ECX,dword ptr [EAX + 0xffffffff8]>=>theScreen

MOV     ECX=>theScreen,dword ptr [ECX]
CMP     dword ptr [ECX + 0x258],0x0

JNZ     LAB_00011d04

LAB_00011cfb
MOV     byte ptr [EBP + local_11],0x0
JMP     LAB_00011e4e

LAB_00011d04
MOV     EAX,dword ptr [EBP + local_28]
MOV     ECX,dword ptr [EAX + 0xffffffff8]>=>theScreen

Decompile: Java_net_godfather_thegodfather_MainService_vn...
1
2 bool Java_net_godfather_thegodfather_MainService_vncConnectReverse
3     (JNIEnv *env,jobject thisObj,jstring host,jint port)
4
5 {
6     char *chars;
7     int iVar1;
8     bool local_11;
9
10    if ((theScreen == 0) || (*(int *) (theScreen + 600) == 0)) {
11        local_11 = false;
12    }
13    else if (host == (jstring)0x0) {
14        local_11 = false;
15    }
16    else {
17        chars = ((*env)->GetStringUTFChars) (env,host,(jboolean *)0x0);
18        if (chars == (char *)0x0) {
19            local_11 = false;
20        }
21        else {
22            iVar1 = rfbReverseConnection(theScreen,chars,port);
23            ((*env)->ReleaseStringUTFChars) (env,host,chars);
24            local_11 = iVar1 != 0;
25        }
26    }
27    return local_11;
```

# Anti-Emulation

```
String locate = Resources.getSystem().getConfiguration()
if (ArrayUtils.contains(this.mw_countriesExcludeList, locate)) {
    finish();
} else if (this.mw_mainWorkClass.isEmulator()) {
} else {
    if (this.mw_mainWorkClass.PRead(this, "key") ==
```



# Common Device Characteristic Checks

|              |                                     |
|--------------|-------------------------------------|
| Fingerprint  | Generic                             |
| Model        | Emulator, Android SDK built for x86 |
| Brand        | generic_x86                         |
| Device       | vbox86p                             |
| Manufacturer | Genymotion, unkown                  |
| Hardware     | Goldfish                            |



# Detecting Malware





# VirusTotal

0.tcp.ngrok.io

12

/ 89

12 security vendors flagged this domain as malicious

Similar Graph API

0.tcp.ngrok.io

ngrok.io

Registrar  
NAMECHEAP INC

Creation Date  
9 years ago

Last Analysis Date  
16 hours ago

Phishing and Other Frauds software-hardware spyware and malware top-1M

Community Score

DETECTION

DETAILS

RELATIONS

COMMUNITY 16 +

Join the VT Community

 and enjoy additional community insights and crowdsourced detections, plus an API key to [automate checks](#).

Security vendors' analysis ⓘ

Do you want to automate checks?

|             |             |         |             |
|-------------|-------------|---------|-------------|
| Antiy-AVL   | ⚠ Malicious | Avira   | ⚠ Malware   |
| BitDefender | ⚠ Phishing  | Certego | ⚠ Malicious |
| CyRadar     | ⚠ Malicious | Dr.Web  | ⚠ Malicious |
| G-Data      | ⚠ Phishing  | Lionic  | ⚠ Malicious |
| Segasec     | ⚠ Phishing  | Sophos  | ⚠ Malware   |
| VIPRE       | ⚠ Phishing  | Webroot | ⚠ Malicious |
| Abusix      | ✅ Clean     | Acronis | ✅ Clean     |

# Yara Rules

```
rule android_rat_androidTester : Android RAT {
  meta:
    author      = "@_lubiedo"
    date        = "2020-12-02"
    description = "Android Tester"
    malver      = "v.6.4.6"
    version     = "1.0"
  strings:
    $str00      = "Android Tester" fullword
    $str01      = "6.4.4" fullword
    $str02      = "device_admin" fullword
    $res03      = "res/drawable/abc_"
    $res04      = "res/layout/abc_"
  condition:
    uint32be(0) == 0x504B0304 // APK file signature
    and filesize < 1MB and (
      all of ($str*) and any of ($res*)
    )
}
```

# Slides and Android RE Challenge



- ▶ Github repo with slides
- ▶ Android RE challenge!
  - ▶ Can you find the C2 in this SpyNote sample?
  - ▶ SHA256:  
5c01f7727c78dea9c89dccf92b01b4c45e69406e  
6462340779401497bf4d4589

<https://github.com/LaurieWired/ReverseEngineeringAndroidMalware>

Thank  
you!



Questions?